10 UALink Switch Requirements

10.1 Overview

UALink Switches serve to relay UALink Requests and Responses between Accelerators.

All switch ports shall connect to Accelerators and Switch-to-switch links shall not be supported. Each Request or Response shall be produced by a Source Accelerator, shall enter the Switch via an ingress Link, shall be relayed by the Switch to a single egress Link, where it shall consumed by a Destination Accelerator. All Response and Request relaying shall be unicast.

A UALink Switch is shall only responsible for the delivery of Requests and Responses between accelerators. A UALink Switch shall not be required to decode the contents of these Requests or Responses beyond that necessary to deliver the Requests and Reponses, norshall the Switch track any Request/Response state.

Inbound DL flits arriving from the Source Accelerator shall be unpacked into TL flits, which shall then be further unpacked into individual Requests and Responses for forwarding between Ports. The Destination Accelerator ID field within each Request or Response shall index into a routing table to determine the egress Station and Port to which it must be forwarded. Once forwarded between ports, outbound Requests and Responses shall be packed into TL flits, which in turn shall be packed into DL flits and sent to the Destination Accelerator.

Each Request or Response shall be routed independently. Multiple Requests or Responses unpacked from the same inbound TL flit do not necessarily route to the same egress Station and Port and multiple Requests and Responses packed into the same outbound TL may have arrived from multiple ingress stations and ports.

The UALink Switch shall contain a complete implementation of the UALink DL and TL, similar to that found in an Accelerator. The nominal interface between TL and switch core shall be the UPLI Originator/Completer interface, although a Switch design may substitute any suitable vendor-defined interface. Any vendor-defined interface replacing UPLI shall mimic any UPLI mandated Originator or Completer behaviors in this Specification, including but not limited to UPLI Drop modes (see 3.1.3) and Port ID field manipulation (See 2.4).

Flow control across an individual link between accelerator and switch, including the required independence of Requests vs Responses, shall be handled by the TL layer logic on each end of the link. Requests or Responses stalled by flow control shall wait within the outbound TL until they are eligible, at which time they may be included in a TL flit being packed and sent. Flow control for Requests or Responses being forwarded between Source and Destination Stations also respects the independence of Requests versus Responses, as does the UPLI interface between TL and switch core.

10.2 Bifurcation support

Link bifurcation feature allows a station to split into multiple independent ports. Each ULS station has four lanes. Each station shall support independent configured as a single four-lane port, two two-lane ports, or four single-lane ports.

10.3 Lossless Request and Response delivery

Except in error cases which explicitly require Requests or Responses to be dropped (see Section 3.1.3), all Requests or Responses shall be relayed through the switch in a lossless manner. Link level Retry (LLR) shall be used to ensure delivery from source Accelerator to the switch ingress

port, and from switch egress port to destination Accelerator. Flow control shall ensure that Requests and Responses are never dropped due to receive-buffer space limitations.

10.4 Non-blocking architecture

Traffic between any one pair of Ports shall operate independently of traffic between any other pair of Ports. However, switch core architecture may share resources across ports within a station or even amongst multiple stations.

Ingress and egress traffic of a port shall operate independently of each other. Ingress Requests shall operate independently of ingress Responses, and egress Requests shall operate independently of egress Responses.

Stalled egress Requests or Responses to one port of a station may block same station egress Requests or Responses headed towards another port of the same station. Similarly, stalled ingress Requests or Responses from one port of a station may block like ingress Requests or Responses from another port of the same station. Under no circumstances shall stalled Requests be permitted to block Responses.

While such blocking is permitted, it should be minimized by switch designs to the extent reasonably possible. Non-blocking architecture is a crucial property of network switch design. Switches should incorporate sufficient buffering at the ingress and/or egress ports to prevent congestion and blocking.

10.5 Forward progress guarantee

All arbitration within the ULS shall be starvation-free, to avoid livelock or starvation cases. This shall include but is not limited to arbitration between ingress ports within a station, arbitration between ingress stations competing for access to an egress station, and arbitration between Requests and Responses for access to available TL flit sectors.

Starvation-free arbitration may include simple round-robin, weighted and/or deficit round-robin, age-based oldest-first, etc.

10.6 Ordering and Virtual Channels

Each port-to-port path through ULS shall follow Request or Response delivery ordering and Virtual Channel handling as defined for UPLI (see 2.7.9). Support for Strict Ordering mode is mandatory, and support for non-Strict Ordering mode is optional. If non-Strict Ordering mode is supported, it shall be selected on a per-port basis, with identical settings being used on ingress and egress ports.

10.7Routing Table Structure

Routing decisions shall be made by looking up the 10-bit Destination Accelerator ID of inbound Requests and Responses in a route table. The number of entries in the table shall equal or exceed the maximum number of supported ports in the Switch with single-lane bifurcation. Each table entry shall contain the following information:

- For each port of a station, an indication of whether to deny routing matching Requests or Responses received by the port, versus allowing the matching Requests or Responses to be routed shall be provided. The Deny setting may be used to prevent routing Switches in a partitioned Physical Switch, and between Virtual Pods. Upon Switch reset, the Deny setting shall apply to all table entries.
- If not denied, an indication of the egress station and port to which the Request or Response shall be routed shall be provided. For example, in a switch with 64 stations, each

bifurcatable into at most four ports, each route table entry shall supply a 6-bit station number and a 2-bit port number.

Where the table contains fewer than 1,024 entries, Requests or Responses whose Destination Accelerator ID value exceeds the capacity of the table shall be denied routing. Requests or Responses which are denied routing shall be silently dropped and shall be logged by the management controller.

10.7.1 Routing Table Instances

The switch shall contain a separate, independently programmable instance of the Routing Table for each station. Where a station contains multiple ports due to bifurcation, all ports within the station shall share the same routing table.

The independent programmability of different routing table instances, and of the independent deny controls for different ports within each bi-furcated station, allow a switch to be subdivided into multiple independent Virtual Switches , serving multiple Virtual Pods.

10.7.2 Egress port reachability

All egress ports of all stations shall be reachable from any ingress port, with no arbitrary restrictions. This shall include allowing route-to-self (a.k.a. U-turn) cases, where the Requests and Responses ingress and egress via the same port.

10.8Configuration

Configurability options are left up to the implementation. The list below is an incomplete but a useful list:

- Accelerator ID value associated with each port.
- Routing table associated with each port, as described in Section (see 10.7)Bifurcation mode for each station.
- For each port, the Strict versus non-Strict ordering mode (see 10.6).
- For each port, whether or not Authentication is enabled, which affects TL flit packing and unpacking.
- Ability to determine which stations are in drop mode, and in the case of port-granular modes, ability to determine which ports are in drop mode.
- Ability to reset drop modes to resume normal operation, with station granular or port granular control, as appropriate.

10.9 UALink Switch Recommendations and Goals

The following subsections are recommendations and goals and are not meant to form UALink switch requirements but are meant to help guide implementors in areas of importance.

10.9.1 Debug

It is recommended that UALink switches support both transaction injection with and without errors towards switch core and towards the UALink. Other vendor defined debug support is allowed, but none is required under this section. UALink switches are recommended to support injection of key errors (i.e. link errors, link down, at least one silicon ECC error) to allow for platform/system level testing of RAS. The Injection, if supported, shall be disabled by default during mission mode and shall have a mechanism to allow enabling injection through a secure FW switch.

10.9.2Latency goals

UALink Switches are recommended to target the following idle and unloaded pin-to-pin latency for a 64 byte Write Request on a 4-lane non bifurcated port with FEC enabled based on size of the switch:

- 128 lane switch: <200ns
- 256 lane switch: <250ns
- 512 lane switch: <300ns

10.9.3 Performance goals

It is recommended that UALink Switch core should maintain a post-FEC line rate of 200Gbps. Switches should be capable of maintaining line rate for pairwise communication as well as concurrent communication across a small group (8-to-32) of accelerators. DL and TL protocol overhead will lower the effective line rate based on the traffic pattern and security protocols.